

Draw It or Lose It
CS 230 Project Software Design Document
Version 3.0

Table of Contents

CS 230 Project Software Design **Error! Bookmark not defined.**

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 3
Evaluation 4
Recommendations 6

Document Revision History

Version	Date	Author	Comments
3.0	08/18/2026	Ricky Bowman	Updated Project Three recommendations and additional changes. Including more detailed technical justifications based on feedback.

Executive Summary

The Gaming Room wants to take Draw It or Lose It beyond Android and make it available to more players on different devices. I think the best way to do this is to turn the game into a web-based application that players can access from Windows, Mac, Linux, Android, and iOS. The Java code that is already being used can still work well for the server side since Java can run on different operating systems. The server would keep track of the main game information while players connect to it through their web browser. This would make the game easier to maintain and would also make it easier to add more users or features later without having to build a completely different version for every device.

Requirements

The main requirements are that the game needs to support multiple games, teams, and players without allowing duplicate names. Everyone also needs to see the same game information even if they are using different devices. On the technical side, the project uses Java, the Entity class for shared information, unique IDs, and one GameService instance to manage the games. Since the goal is to make the game web based, it also needs to work in browsers on Windows, Mac, Linux, Android, and iOS. The server should be able to handle more players as the game grows.

Design Constraints

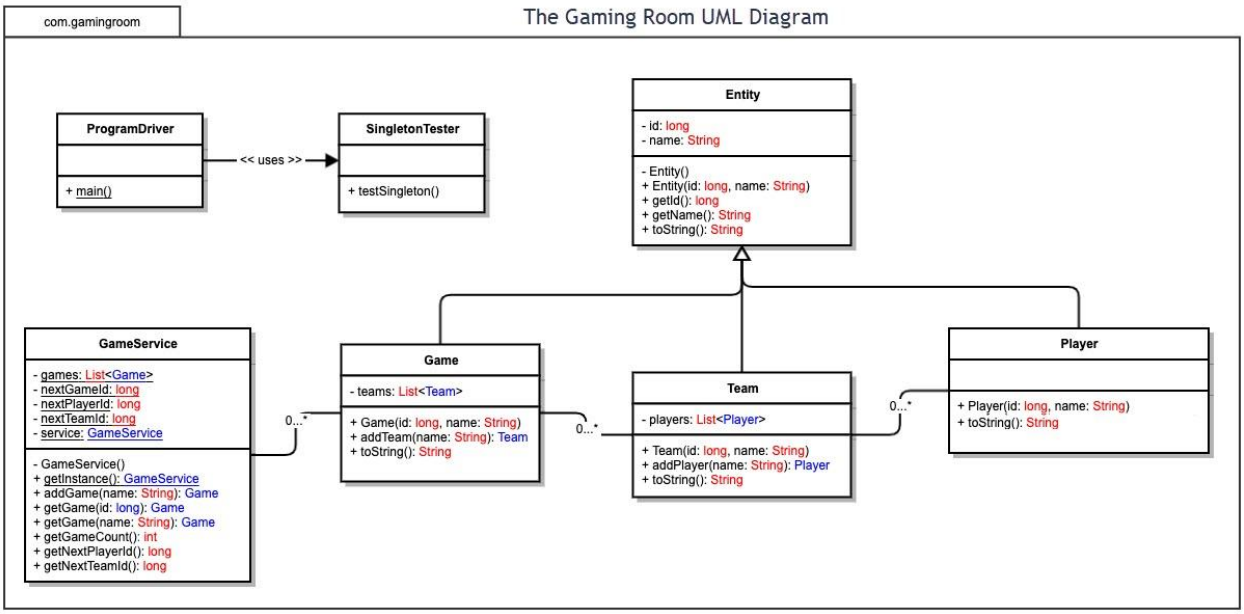
One of the biggest design constraints is making sure the game works the same way for people using different operating systems and devices. Players could be connected at the same time using different browsers, screen sizes, and internet connections. Because of that, the game needs a responsive web design and one central place for the game data. The server also must handle multiple users at once without creating conflicting information. Other things that need to be considered are browser compatibility, slower internet connections, temporary outages, login security, and keeping information safe while it is being sent between the player and the server.

System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The UML diagram shows how the different parts of the game connect to each other. GameService manages the games, each Game can have teams, and each Team can have players. Game, Team, and Player all use the Entity class for shared information like the ID and name. This is an example of inheritance because the same basic information does not have to be written again in every class. The classes also keep their own data and methods together, which is an example of encapsulation. GameService uses the Singleton pattern so there is only one instance managing the games. The program can also check the existing lists before adding something new, which helps stop duplicating game, team, or player names.



Evaluation

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	Mac can run a Java web server and has a stable Unix-based system. It also supports most of the tools that would be needed for this project. The downside is that using Mac as the main server would be more limiting because macOS is tied to Apple hardware. It could work, but I do not think it makes as much sense as Linux for a game that may need to grow and support a lot of players.	Linux would be my first choice for the server. It works very well with Java, databases, web servers, and cloud hosting. It is also known for being stable and can handle a lot of users when it is set up correctly. Another advantage is that many Linux versions are free, so there would not normally be an extra operating system license cost. The main drawback is that someone on the team would need to be comfortable working with Linux.	Windows Server could also host the game and would probably be familiar to a lot of developers. It supports Java, databases, and the other tools needed for the project. The biggest downside is the possible licensing cost. Since the game does not really need anything that is Windows-only, I do not see a strong reason to pay more for Windows Server when Linux can do the same job.	I would not use a phone or tablet as the actual game server. Mobile devices have limited battery life, changing internet connections, and restrictions from Android or iOS. They make much more sense as clients that connect to the main server. The benefit is that players could access the game from almost anywhere without needing the game server to run on their device.

Client Side	For Mac users, the easiest option would be to let them play through a web browser instead of making a separate Mac application. The team would still need to test Safari and other browsers to make sure everything displays and works correctly. This saves development time because most of the same web code can be used on every desktop platform.	Linux users could also play through the same browser-based version. This means there would not need to be a separate Linux client. The main work would be testing common browsers and making sure the website communicates correctly with the server. Using the same HTML, CSS, and JavaScript across platforms would help keep development time and cost lower.	Windows users can use the same web application through browsers such as Edge, Chrome, or Firefox. This is easier than building a separate Windows desktop program. The team would still need to test things like screen scaling, keyboard and mouse controls, and browser compatibility, but most of the client code could stay the same.	Mobile devices would take some extra testing because phones and tablets have different screen sizes and use touch controls. Android and iOS browsers can also behave a little differently. A responsive website would be the best starting point because it can adjust to different screen sizes. This would take less time and money than building and maintaining separate Android and iOS apps.
Development Tools	For Mac development, Java can still be used for the server and HTML, CSS, and JavaScript can be used for the web client. Tools like IntelliJ IDEA, Eclipse, Visual Studio Code, Git, and Maven or Gradle would all work. Xcode would mainly be needed if the team later decided to build something specifically for Apple devices. Many of these tools have free versions.	Linux supports the main tools needed for this project, including Java, IntelliJ IDEA, Eclipse, Visual Studio Code, Git, Maven or Gradle, Docker, MySQL, and PostgreSQL. A lot of these tools are free or open source. This helps keep costs down, but the team should have someone who knows how to manage a Linux server.	Windows has plenty of development options for Java and web development. IntelliJ IDEA, Eclipse, Visual Studio Code, Git, Maven or Gradle, Docker, MySQL, and PostgreSQL can all be used. The tools themselves can often be used for free, but Windows Server or paid versions of some software could add extra costs.	For mobile devices, the main client can be made with HTML, CSS, and JavaScript and tested on real Android and iOS devices. If native apps are added later, Android Studio with Java or Kotlin could be used for Android, while Xcode and Swift would be used for iOS. Native apps would take more time and probably require developers with more specialized experience, so I would start with the web version.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. Operating Platform: I would go with Linux for the production server for Draw It or Lose It. I think it makes the most sense because it is reliable, secure, works well with Java, and does not add the same licensing cost that Windows Server could. It is also commonly used for web servers, databases, and cloud hosting, so it gives The Gaming Room more room to grow later. macOS could technically work, but it is tied more closely to Apple hardware, which makes it less flexible for a server. With Linux handling the server side, players would still be able to use Windows, Mac, Linux, Android, or iOS because they would connect through a web browser. This keeps the setup simpler because the team would not have to build and maintain a completely different version of the game for every operating system.

2. Operating Systems Architectures: I would use a 64-bit Linux server architecture. Linux handles the main system jobs like processes, memory, networking, files, devices, and user permissions through the kernel. The Java game would run as its own application instead of needing full access to everything on the server, which is better for both stability and security. Using 64-bit Linux also gives the server access to much more memory than a 32-bit system, which would be useful if the game gets more players and has more games running at the same time. Java also helps with compatibility because the JVM lets Java code run on different systems without rewriting the whole program for each type of hardware. Another reason I like this setup is that Linux works well with virtual machines and containers. If The Gaming Room needs more server capacity later, they could add more instances without having to completely redesign the application.

3. Storage Management: For information that needs to stay saved, I would use a relational database like PostgreSQL or MySQL. The game has information such as games, teams, players, and possibly user accounts later, and all of these have relationships with each other. A relational database would make that information easier to organize and would help keep the data consistent when multiple people are using the game at once. For the Linux server files, I would use a file system such as ext4. This could store the application files, settings, logs, and other files needed by the server. The database should also be backed up regularly, and I would keep important backups somewhere separate from the main server. That way, if the server or database has a problem, there is a better chance of recovering the game information instead of losing it.

4. Memory Management: Linux uses virtual memory and paging to manage how RAM is shared between programs. Each running process gets its own memory space, which helps keep one program from interfering with another one. If the server starts running low on physical RAM, Linux can move less-used information to swap space, but I would not want the game relying on swap too much because it is slower than RAM. Java also has its own memory management through the JVM. The JVM keeps track of Java objects and uses garbage collection to clear objects that are not being used anymore. This is important for Draw It or Lose It because more players mean more game objects, teams, requests, and active sessions being stored in memory. I would keep an eye on memory and CPU usage as the game grows. Session timeouts and connection limits could also help stop old or abandoned sessions from continuing to use resources.

5. Distributed Systems and Networks: I would use a client-server setup where players connect to the Linux server through their web browser. The connection would use HTTPS, and web APIs could be used to send game information between the browser and the server. I think the server should be the main source for the game state because that helps make sure everyone is seeing the same information even when they are using different devices. There are a lot of parts that depend on each other in this type of system, including the player's device, browser, internet connection, DNS, application server, database, and firewall. If one of these has a problem, it can affect the game. For example, an internet outage could disconnect a player, while a database problem could stop the server from saving or loading game information. Because of that, I would include error messages, timeouts, logging, backups, health checks, and retrying requests when it is safe to do so. If the game becomes much larger, more than one application server could also be used behind a load balancer. This would make it easier to handle more players and would keep one server failure from taking down the whole application.

6. Security: Security is important because players would be sending information over the internet and the server may eventually store account information. I would require HTTPS with TLS so information is encrypted while it travels between the browser and the server. Passwords should never be saved as plain text and should instead use secure password hashing. The server should also check and validate information coming from the client instead of automatically trusting every request. Users and server processes should only have access to what they actually need. Linux user and group permissions can help control access to application files, settings, and logs. I would also use firewall rules so only the ports the game needs are open. Keeping Linux, Java, and other dependencies updated would help fix known security problems. Login attempts, errors, and other important activity should be logged so unusual behavior can be noticed. Database passwords and other private information should also be kept out of the source code. These steps give The Gaming Room several layers of protection instead of depending on only one security feature.